

JAVA INTERVIEW PREPARATION

CHAPTER 29

GARBAGE COLLECTION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Garbage Collection: Reachability, Generations, G1, ZGC, and Evidence-Based Tuning

Senior / Lead Java Developer Reading Chapter

Garbage collection automates reclamation, not capacity planning. A collector must find unreachable objects, recover space, compact or manage fragmentation, and coordinate with application threads. Different collectors trade throughput, pause time, footprint, and CPU overhead. Senior engineers begin with service-level goals and GC evidence rather than copying a list of flags from another application.

Reachability, Not Scope

An object becomes eligible for collection when no GC-root path can reach it. Leaving a source-code block does not guarantee immediate reclamation, and System.gc does not promise that a particular object will be collected.

```
roots -> live graph      unreachable graph
      A -> B              X -> Y
      |                   ^ |
      +-> C                +-+
```

X and Y can be collected despite their cycle. A, B, and C remain live as long as the root path remains.

Collectors normally use tracing phases conceptually related to marking reachable objects, reclaiming unreachable space, and sometimes relocating live objects. Exact algorithms differ.

The Generational Hypothesis

Most applications create many objects that die young and fewer that survive for a long time. Generational collectors exploit this pattern by collecting young allocations frequently and long-lived regions less often.

```
allocation -> young objects -> survivors -> older objects
              many die here   some age      fewer remain
```

Generational is a collector design, not a Java language promise. Object age normally means surviving collections, not wall-clock age. Promotion and region rules are implementation-specific.

Allocation and Thread-Local Buffers

HotSpot can allocate many small objects quickly using thread-local allocation buffers, or TLABs. A thread often advances a pointer within its current buffer without a global heap lock.

```
young region
+-- Thread A TLAB: [used...free]
+-- Thread B TLAB: [used...free]
```

Fast allocation does not make allocation free. Objects increase memory bandwidth, GC scanning, copying, and retained-state pressure. Optimize allocation only after profiling; short-lived objects can be cheaper than complex pooling that raises retention and contention.

Stop-the-World and Concurrent Work

During a stop-the-world pause, application threads reach a safepoint and stop while the JVM performs required work. Concurrent collectors move substantial work alongside application execution, but still have some pauses.

```
application: ===|pause|=====|pause|=====
GC concurrent:      ----marking-----
```

"Concurrent" does not mean pause-free, and "low latency" does not mean zero CPU cost. Concurrent work competes for CPU and memory bandwidth and may require headroom to finish before allocation exhausts available space.

Collector Goals

```
throughput: percentage of time doing application work
latency:    pause and response-time distribution
footprint:  heap plus collector metadata and headroom
capacity:   allocation rate and live-set volume supported
```

These goals conflict. A collector optimized for short pauses can use additional barriers, threads, metadata, and headroom. A small batch job may prefer maximum throughput; an interactive API may prioritize tail latency.

Serial and Parallel Collectors

The Serial collector performs collection work with a single GC thread and can suit small heaps or constrained environments. The Parallel collector uses multiple GC threads and emphasizes throughput, usually accepting stop-the-world pauses.

```
Serial:  one GC worker, simple, small-scale use
Parallel: multiple GC workers, throughput-oriented pauses
```

Do not dismiss them as obsolete. Workload, heap size, CPU limits, and latency requirements determine fit. A short-lived batch process may perform well with a throughput collector.

G1: Region-Based Balanced Collection

G1 divides the heap into equal-sized regions rather than requiring one physically contiguous young generation followed by one old generation.

```
G1 heap regions
[E][E][Y][O][O][H][H][F][Y][O]
E empty, Y young, O old, H humongous, F free
```

G1 performs young-only collections and later mixed collections that include selected old regions with substantial reclaimable space. It tracks cross-region references using remembered structures. G1 is the default collector on most Java 21 HotSpot hardware and operating-system configurations.

`-XX:MaxGCPauseMillis` is a pause-time goal, not a hard deadline. G1 selects work to attempt to meet the goal while preserving progress. Setting an unrealistically tiny target can increase overhead and reduce throughput.

Concurrent Marking and Mixed Collections

G1 begins concurrent marking to estimate liveness across old regions. After marking, mixed collections gradually reclaim selected old regions alongside young regions.

```
young collections -> concurrent mark -> remark -> cleanup
                                     -> mixed collections
```

If marking starts too late or the application allocates faster than G1 can recover space, G1 may experience evacuation failure or fall back to expensive full collection behavior. Heap occupancy, allocation rate, marking duration, and reserve headroom must be considered together.

Humongous Objects

In G1, an object larger than a collector-defined fraction of a region is treated as humongous and occupies one or more contiguous regions.

Large byte arrays, image buffers, serialized payloads, and oversized caches can create fragmentation and special collection pressure.

```
normal objects share regions
humongous array requires consecutive region capacity
```

Do not tune region size first. Identify allocation sites and ask whether payloads can be streamed, chunked, bounded, or avoided. If the objects are legitimate, collector and region configuration may then be evaluated with evidence.

ZGC and Low-Latency Collection

ZGC performs most marking, relocation, and reference processing concurrently and is designed for very low pause times across large heaps. It uses colored-pointer or metadata techniques and load barriers in HotSpot implementations.

Java 21 introduced Generational ZGC as a separate mode while the existing non-generational ZGC remained available. Exact defaults and option status are JDK-version-specific, so production documentation must name the target JDK.

ZGC needs CPU and memory headroom to perform concurrent work faster than the application allocates. A low-pause collector cannot compensate for a live set that nearly fills the configured memory limit or a container with no native headroom.

Reference Types and Cleanup

Strong references keep objects ordinarily reachable. Soft, weak, and phantom references provide different reachability semantics and are processed by the collector.

Weak references can support canonical mappings where keys should not remain alive solely because of the map. Soft references are not a dependable primary cache-sizing policy; reclamation timing is controlled by the JVM and can cause latency spikes. Phantom references plus a ReferenceQueue can support post-reachability cleanup coordination.

Prefer explicit resource lifecycle with try-with-resources. GC manages memory reachability, not timely closure of files, sockets, transactions, or permits.

Finalization is deprecated for removal and has unpredictable timing and security and performance problems. Use AutoCloseable, Cleaner only when a safety net is genuinely required, and explicit ownership.

Reading GC Logs

Unified logging provides collector evidence.

```
-Xlog:gc*:file=gc.log:time,uptime,level,tags
```

Useful observations include:

- Pause type, duration, and frequency.
- Heap occupancy before and after collection.
- Young and old region changes.
- Concurrent-cycle start and completion.
- Allocation or promotion pressure.
- Humongous allocation activity.
- Full collection, evacuation failure, or allocation stall.
- CPU time used by GC phases.

One long pause is not the same as sustained GC pressure. Correlate logs with request latency, allocation profiles, CPU throttling, container events, and traffic.

Live Set, Allocation Rate, and Headroom

The live set is the memory that remains reachable after effective collection. Allocation rate is how quickly new memory is requested. Both matter.

```
large allocation rate + small stable live set -> frequent young collection  
growing post-GC live set                    -> retention growth or workload growth  
live set near heap maximum                  -> little recovery headroom
```

A larger heap can reduce collection frequency but may increase worst-case work and process footprint. A smaller heap can fail during bursts. Size from measured steady-state live set, traffic bursts, collector needs, and container headroom.

Tuning Workflow

1. Define latency, throughput, footprint, and availability goals. 2. Confirm collector, JDK build, heap limits, container CPU and memory. 3. Capture GC logs and JFR under representative load. 4. Measure allocation rate, post-GC live set, pause percentiles, and CPU. 5. Find application causes such as unbounded retention or humongous allocation. 6. Change one high-value setting or design at a time. 7. Load test and compare against the same baseline.

For G1, official guidance generally recommends beginning with defaults, setting a maximum heap, and adjusting the pause goal only when evidence requires it. Large collections of inherited tuning flags can disable useful ergonomics and encode assumptions from another JDK.

Production Perspective

High GC frequency can be the symptom of an allocation storm, undersized heap, memory leak, or workload spike. Long pauses can involve a large live set, reference processing, humongous objects, CPU throttling, or full collection. A low heap percentage does not rule out native-memory pressure.

Do not force System.gc as a routine repair. Diagnose why application or library code requests it, and understand whether the selected collector disables, honors, or handles explicit GC concurrently under the target configuration.

Interview-Ready Summary

GC traces reachability from roots and reclaims unreachable memory. Generational designs exploit the tendency of many objects to die young. Allocation can be fast through TLABs, but allocation rate and retained live set still drive collector work.

G1 uses regions, young and mixed collections, concurrent marking, and pause-time goals. ZGC performs most work concurrently for low latency and requires headroom. Collector selection and tuning trade throughput, latency, footprint, and CPU. Start from goals, logs, JFR, allocation profiles, and the post-GC live set rather than folklore flags.

Revision Notes

- Eligibility for collection depends on reachability from GC roots.
- Java GC is not reference counting; unreachable cycles are collectible.
- Generational designs exploit high mortality among young objects.
- TLABs make many allocations cheap but not free.
- Stop-the-world pauses suspend application execution.
- Concurrent collectors still have pauses and consume CPU.
- Throughput, latency, footprint, and headroom trade against one another.
- Serial uses one GC worker; Parallel emphasizes throughput with multiple workers.
- G1 divides the heap into regions.
- G1 performs young-only and mixed collections.
- G1 is the default on most Java 21 HotSpot configurations.
- MaxGCPauseMillis is a goal, not a deadline.
- G1 concurrent marking estimates old-region liveness.
- Humongous objects occupy special region allocations.
- Java 21 offers existing ZGC and a generational ZGC mode.
- Low-pause collection still needs CPU and memory headroom.
- Soft references are not a complete cache policy.
- GC does not replace timely resource closure.
- Finalization is deprecated for removal.
- Read pause duration together with occupancy and application latency.
- Post-GC live-set growth is stronger leak evidence than sawtooth heap usage.
- Start tuning from defaults, evidence, and service-level goals.